Lecture–3: Introduction to DataFrames in Pandas

Topics Covered:

1. Introduction to DataFrames
2. Creating DataFrame from List, Tuple, Set and Dictionary
3. Converting a Series to DataFrame
4. Assigning Names to Columns in DataFrame

(According to your Placement Oriented DA syllabus – Lecture 3)

---

1. Introduction to DataFrames

What is a DataFrame?

A **DataFrame** is a **2-dimensional labeled data structure** in Pandas.

Think of it as:

- Excel Sheet
- SQL Table
- Database Table

having:

Rows + Columns

Example:

Name	Age	Salary
Ram	22	40000
Rita	25	50000
Aman	24	45000

This entire table is a DataFrame.

Why DataFrames are Important?

Companies store data as tables:

Amazon:

Customer	Purchase
Ram	1000
Rita	2000

Hospitals:

Patient	Disease
A	Fever

Schools:

Student	Marks

DataFrame handles all such tabular data.

Real-Life Analogy

Series (1D)

One subject marks:

Math Marks:
80
90
70

DataFrame (2D)

Complete report card:

Name	Math	Science
Ram	80	90
Rita	85	95

Hence:

Series → One column

DataFrame → Multiple columns

Difference Between Series and DataFrame

Feature	Series	DataFrame
Dimension	1D	2D
Structure	Single column	Multiple columns
Example	Marks	Student Report
Index	Yes	Yes
Columns	No	Yes

Creating a DataFrame

Syntax:

pd.DataFrame(data)

Import Pandas First

Every time:

import pandas as pd

---

2. Creating DataFrame from Different Data Structures

We can create DataFrame using:

1. List
2. Tuple
3. Set
4. Dictionary
5. Series

A) Create DataFrame from List

Suppose:

Student names:

students=["Ram","Rita","Aman"]

Convert to DataFrame:

import pandas as pd

students=["Ram","Rita","Aman"]

df=pd.DataFrame(students)

print(df)

Output:

```
      0
0   Ram
1  Rita
2  Aman
```

Explanation:

Pandas automatically creates:

Row index

Column name

List of Lists → Multiple Columns

Example:

```
data=[
["Ram",22],
["Rita",25],
["Aman",24]
]

df=pd.DataFrame(data)

print(df)
```

Output:

```
0      1
Ram   22
Rita  25
Aman  24
```

Add Column Names

```
df=pd.DataFrame(
data,
columns=["Name","Age"]
)

print(df)
```

Output:

Name	Age
Ram	22
Rita	25
Aman	24

B) Create DataFrame from Tuple

Tuple:

```
data=(
("Ram",22),
("Rita",25),
("Aman",24)
)
```

Convert:

```
df=pd.DataFrame(
data,
columns=["Name","Age"]
)

print(df)
```

Output:

Name	Age
Ram	22
Rita	25
Aman	24

Difference:

Tuple is immutable

List is mutable

C) Create DataFrame from Set

Set:

data={"Ram","Rita","Aman"}

Convert:

df=pd.DataFrame(data)

print(df)

Possible output:

0
Ram
Aman
Rita

Important:

Sets are unordered.

Output order may change.

Interview Question:

Why avoid sets?

Answer:

Because sets don't preserve order.

D) Create DataFrame from Dictionary

Most important method in real projects.

Dictionary:

```
student={
"Name":["Ram","Rita","Aman"],
"Age":[22,25,24]
}
```

Convert:

```
df=pd.DataFrame(student)

print(df)
```

Output:

Name	Age
Ram	22
Rita	25
Aman	24

Explanation:

Keys become:

Column names

Values become:

Column data

Example: Employee Dataset

```
employee={

"Name":["A","B","C"],

"Salary":[40000,50000,60000],
```

```
"City":["Delhi","Mumbai","Pune"]

}

df=pd.DataFrame(employee)

print(df)
```

Output:

Name	Salary	City
A	40000	Delhi
B	50000	Mumbai
C	60000	Pune

This resembles company datasets.

Summary: Creating DataFrames

Structure	Syntax
List	pd.DataFrame(list)
Tuple	pd.DataFrame(tuple)
Set	pd.DataFrame(set)
Dictionary	pd.DataFrame(dict)

---

3. Converting Series to DataFrame

Suppose Series:

```
s=pd.Series(
[10,20,30]
)

print(s)
```

Output:

0 10
1 20
2 30

Convert to DataFrame:

Method 1:

df=pd.DataFrame(s)

print(df)

Output:

0

10

20

30

Method 2: Using to_frame()

Recommended

df=s.to_frame()

print(df)

Output:

0

10

20

30

Give Column Name

```
df=s.to_frame(name="Marks")

print(df)
```

Output:

Marks
10
20
30

Very useful in projects.

Example: Student Marks

Series:

```
marks=pd.Series(
[80,90,70]
)
```

Convert:

```
df=marks.to_frame(
name="Math"
)

print(df)
```

Output:

Math
80
90
70

---

4. Assigning Names to Columns in DataFrame

Column naming is extremely important.

Bad:

|0|1|

Good:

|Name|Age|

Method 1: Assign During Creation

Example:

```
data=[
["Ram",22],
["Rita",25]
]

df=pd.DataFrame(
data,
columns=["Name","Age"]
)

print(df)
```

Output:

Name	Age
Ram	22
Rita	25

Recommended method.

Method 2: Rename Existing Columns

Suppose:

```
df=pd.DataFrame(
[
["Ram",22]
]
)
```

print(df)

Output:

|0|1|

Rename:

```
df.columns=[
"Name",
"Age"
]
```

print(df)

Output:

| Name | Age |

Method 3: Rename Selected Columns

Use:

```
df.rename(
columns={
"Age":"Student_Age"
}
)
```

Example:

Before:

| Name | Age |

After:

| Name | Student_Age |

Method 4: Rename All Columns

```
df.columns=[
"Student_Name",
```

"Student_Age"
]

Example: Employee Dataset

Original:

employee={

"name":["A","B"],

"salary":[50000,60000]

}

df=pd.DataFrame(employee)

Rename:

df.columns=[
"Employee_Name",
"Employee_Salary"
]

Output:

Employee_Name	Employee_Salary
A	50000
B	60000

Common Interview Questions

Q1 What is DataFrame?

A 2-dimensional labeled data structure.

Q2 Difference between Series and DataFrame?

Series:

Single column

DataFrame:

Multiple columns

Q3 Which structure is mostly used in projects?

Answer:

Dictionary → DataFrame

Q4 Why are sets not preferred?

Answer:

Because sets are unordered.

Q5 How to convert Series to DataFrame?

pd.DataFrame(series)

OR

series.to_frame()

Q6 How to rename columns?

df.columns=[]

or

df.rename()

Lecture–3 Summary

Students should know:

✓ What is DataFrame

✓ Difference:

Series vs DataFrame

✓ Create DataFrame from:

- List
- Tuple
- Set
- Dictionary

✓ Convert Series to DataFrame

✓ Assign column names

✓ Rename columns

Practice Questions

Question 1

Create DataFrame:

Name
Ram
Rita
Aman

using list.

Question 2

Create DataFrame from tuple:

("Ram",22)

("Rita",25)

Question 3

Create DataFrame from dictionary:

Name:
A,B,C

Salary:
10000,20000,30000

Question 4

Convert:

s=pd.Series([10,20,30])

to DataFrame.

Question 5

Rename columns:

0 → Name

1 → Age